Reproducing and Improving TypiClust

Lucas Lobo - k23075501

1 Introduction

Deep learning requires large labelled datasets, yet annotation is costly. Active Learning (AL) mitigates this by iteratively selecting unlabelled examples, but traditional uncertainty-based methods rely on a trained model, creating a cold start problem. Hacohen et al. [1] identified that the optimal AL strategy depends on budget size: typical examples should be queried first, with uncertainty-based methods taking over once sufficient labels exist. They proposed TypiClust, which selects typical points from K-Means clusters in a self-supervised representation space. However, typicality alone can produce redundant selections. We reproduce TypiClust on CIFAR-10 and propose a hybrid modification incorporating diversity-aware selection.

2 Methodology

2.1 TypiClust Pipeline

Our implementation follows the TPC_{RP} variant for initial pool selection ($L_0 = \emptyset$). Following the paper, we train a SimCLR [2] encoder (ResNet-18, 200 epochs) from scratch on CIFAR-10, producing 512-d embeddings. We simplify the training setup: Adam optimiser (vs. LARS), batch size 256 (vs. 4096), and no cosine LR schedule, reducing training to ∼2 hours on a single GPU versus the paper's TPU-scale setup. This narrows the TPC_{RP} accuracy gap by ∼4% vs. the paper (44.4% vs. ∼48%) while the random baseline remains comparable (36.9% vs. ∼38%), confirming that relative method ordering is preserved. K-Means with $K = B$ partitions embeddings into B clusters, and from each the point with the highest typicality is selected:

$$\text{Typicality}(x) = \left(\frac{1}{k} \sum_{x_i \in \text{KNN}(x)} \|x - x_i\|_2 \right)^{-1} \tag{1}$$

where $k = 20$ neighbours. Higher values indicate points more central to the data distribution. Selection quality is evaluated via a linear probe (logistic regression on frozen embeddings), isolating the effect of sample selection from representation quality.

2.2 Proposed Modification

Even though TypiClust demonstrates strong performance in low-budget regimes, selecting points based purely on typicality can still choose examples that are spatially close in the embedding space, under representing regions of the data distribution. This motivates incorporating spatial diversity, a principle exploited by CoreSet [4]. We propose the following modifications:

Hybrid Score Formula. We replace the pure typicality criterion with a hybrid score:

$$\text{score} = \alpha \cdot \hat{t} + (1 - \alpha) \cdot \hat{d} \tag{2}$$

where $\hat{t}$ is the normalised typicality and $\hat{d}$ is the normalised minimum distance to already-selected points, both scaled to $[0, 1]$ within each cluster. This balances representativeness with spatial diversity, reducing redundancy among selected examples. Selection proceeds sequentially: the most typical point overall is chosen first, and each subsequent point is scored considering distance from already-selected points.

Dimensionality Reduction. High-dimensional embeddings (512-d) can suffer from the curse of dimensionality [5], where distance metrics become less meaningful. We apply PCA and UMAP [3] to reduce dimensionality before clustering. UMAP is chosen over alternatives such as t-SNE because it preserves both local and global structure and supports transforming unseen test data, which is necessary for consistent evaluation. However, UMAP introduces additional hyperparameters (*n_components*) and stochasticity.

Alpha Tuning. The parameter α controls the trade-off between typicality and diversity: $\alpha = 1$ recovers the original TypiClust selection, while $\alpha = 0$ selects purely for maximum distance. We sweep α across $[0, 1]$ to find the optimal balance.

3 Results

3.1 Paper Results

Table 1 summarises key results from [1] on CIFAR-10 ($B = 10$). Both TypiClust variants outperform all baselines in the low-budget regime. In the semi-supervised setting with FlexMatch, TypiClust reaches 93.2% accuracy (+39.4% over random). As budget grows, uncertainty methods eventually surpass TypiClust, confirming the phase transition.

Strategy	Supervised	SS Embed.	Semi-sup.
Random	∼18%	∼38%	53.8%
Uncertainty	∼15%	∼35%	–
TPC_{RP}	**∼28%**	**∼48%**	**93.2%**

Table 1: Paper results on CIFAR-10 ($B = 10$, first AL round). Supervised and SS Embedding values estimated from [1] Figs. 4–5.

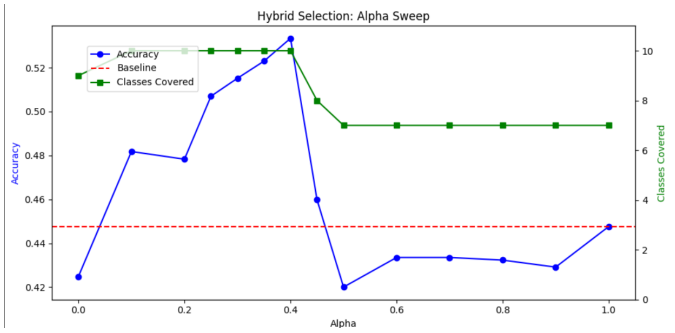

Figure 1: Hybrid selection on CIFAR-10 ($B = 10$): accuracy and class coverage vs. α. Dashed line shows baseline TypiClust.

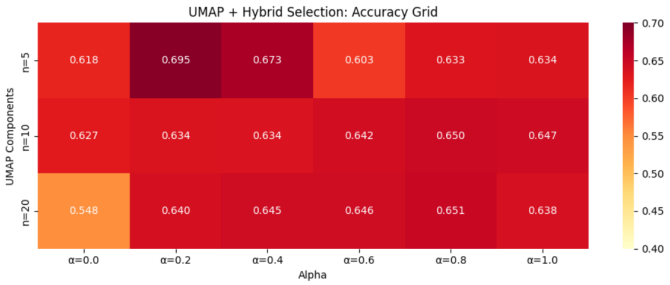

Figure 2: UMAP + hybrid accuracy grid on CIFAR-10 ($B = 10$) across UMAP dimensions and α.

3.2 Our Results

We reproduce TPC_{RP} on CIFAR-10 with $B = 10$. To address the single-run limitation, we evaluate each method across 10 K-Means random seeds and report mean accuracy ± standard deviation. A random baseline is established over 30 seeds, yielding $36.92\% \pm 6.75\%$.

The baseline TypiClust reproduction achieves $44.42\% \pm 0.42\%$, covering 7 out of 10 classes on average. While this confirms that typicality-based selection outperforms random sampling, the limited class coverage motivates our modifications. The alpha sweep (Figure 1) shows that introducing hybrid selection ($\alpha = 0.4$) raises accuracy to 53.34% with full class coverage (10/10), demonstrating that diversity substantially reduces redundancy. Adding PCA dimensionality reduction ($n = 10$, $\alpha = 0.55$) improves accuracy to 57.75%, though class coverage drops to 8, suggesting that PCA's linear projection can merge semantically distinct clusters, a trade-off between tighter clusters and class separation. Figure 2 shows that the best result is UMAP ($n = 5$) with hybrid selection ($\alpha = 0.2$): $65.78\% \pm 1.71\%$ covering 9.6 classes on average, an improvement of +21.36% over baseline (Figure 3). This configuration favours diversity ($\alpha = 0.2$), indicating that in UMAP's compact space, spatial separation becomes the dominant factor. The non-zero variance across seeds reflects UMAP's stochastic nature, though the improvement remains highly significant.

We also evaluate over-clustering ($K > B$) across multipliers (2×–10×) with both PCA and UMAP preprocessing. Over-clustering consistently degrades performance, validating the paper's $K = B$ design.

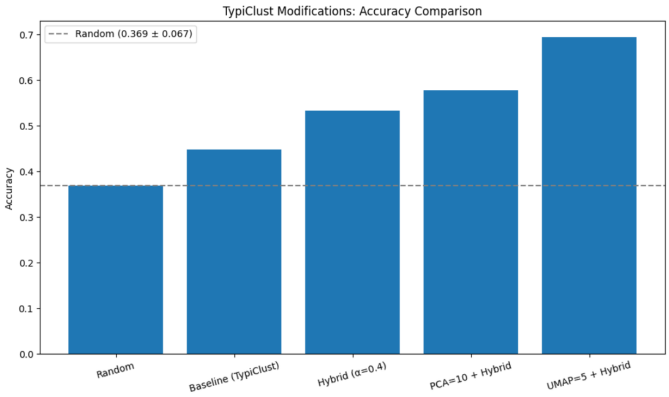

Figure 3: Accuracy comparison across all selection methods on CIFAR-10 ($B = 10$). Error bars show ±1 std over 10 seeds.

Method	Accuracy	Classes
Random (30 seeds)	$(36.92 \pm 6.75)\%$	–
TypiClust (baseline)	$(44.42 \pm 0.42)\%$	7
Hybrid ($\alpha = 0.4$)	$(53.34 \pm 0.00)\%$	10
PCA-10 + Hybrid	$(57.75 \pm 0.00)\%$	8
UMAP-5 + Hybrid	$(\mathbf{65.78 \pm 1.71})\%$	9.6

Table 2: Selection strategies on CIFAR-10 ($B = 10$), averaged over 10 seeds.

4 Statistical Analysis

We compare each method (10 seeds) against the random baseline (30 seeds) using two-sample t-tests. All achieve significance: baseline ($t = 3.42$, $p = 0.0015$, $d = 1.57$), hybrid ($t = 7.50$, $p < 0.001$, $d = 3.44$), PCA + hybrid ($t = 9.52$, $p < 0.001$, $d = 4.37$), UMAP + hybrid ($t = 13.05$, $p < 0.001$, $d = 5.86$). The progressively increasing Cohen's d values confirm that each modification adds a practically meaningful improvement, not just statistical noise. With Bonferroni correction ($\alpha_{\text{adj}} = 0.0125$), all remain significant. Hybrid and PCA show zero variance due to K-Means stability ($n_init = 10$); UMAP variance ($\sigma = 1.71\%$) reflects its stochastic initialisation.

5 Conclusion

TypiClust's typicality-based selection outperforms random sampling in the low-budget regime, consistent with [1]. Our hybrid modification improves accuracy from 44.42% to 65.78% with UMAP (+21.36%), suggesting the original pipeline underutilises embedding geometry. Over-clustering is detrimental, validating $K = B$.

Limitations & Future Work. Evaluation is limited to CIFAR-10 ($B = 10$) with a linear probe. UMAP adds cost and stochasticity. Future work: multi-round AL, semi-supervised integration, and larger datasets.

Code: `https://github.com/LucasPRLobo/ml-cw2`

References

[1] G. Hacohen, A. Dekel, and D. Weinshall. Active learning on a budget: Opposite strategies suit high and low budgets. *ICML*, PMLR 162, 2022.

[2] T. Chen, S. Kornblith, M. Norouzi, and G. Hinton. A simple framework for contrastive learning of visual representations. *ICML*, pp. 1597–1607, 2020.

[3] L. McInnes, J. Healy, and J. Melville. UMAP: Uniform manifold approximation and projection for dimension reduction. *arXiv:1802.03426*, 2018.

[4] O. Sener and S. Savarese. Active learning for convolutional neural networks: A core-set approach. *ICLR*, 2018.

[5] C. C. Aggarwal, A. Hinneburg, and D. A. Keim. On the surprising behavior of distance metrics in high dimensional space. *ICDT*, pp. 420–434, 2001.

Additional Plots

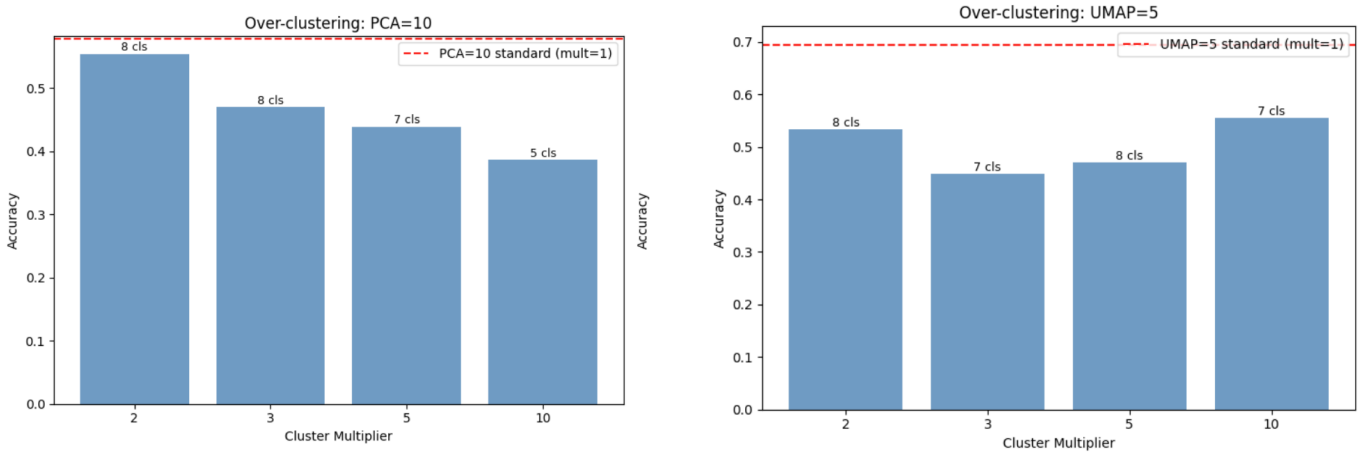

Figure 4: Over-clustering ablation for PCA-10 (left, $\alpha = 0.55$) and UMAP-5 (right, $\alpha = 0.2$). Dashed lines: standard $K = B$ result. Over-clustering consistently degrades performance.

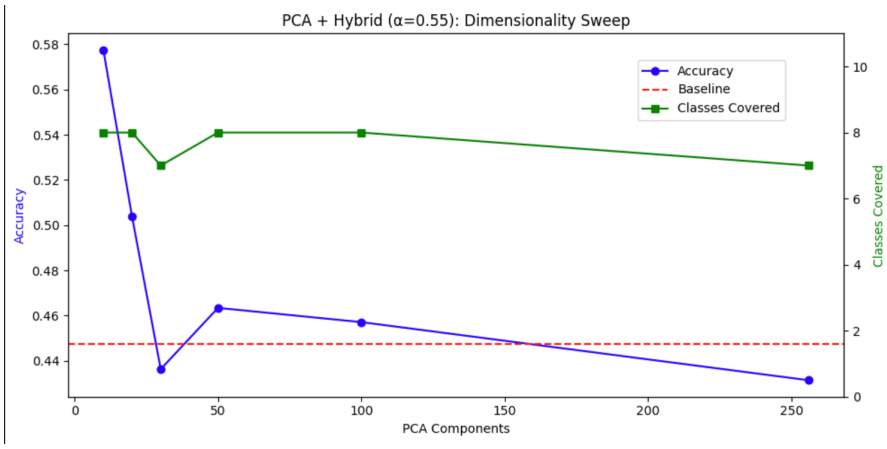

Figure 5: PCA + hybrid ($\alpha = 0.55$): accuracy and class coverage across PCA dimensions.

SimCLR_implementation

March 17, 2026

1 SimCLR implementation

[11]:
```
import torch
import torchvision.transforms as transforms
import torch.nn as nn
import torchvision
import torchvision.models as models

from torch.utils.data import DataLoader
from tqdm import tqdm
```

1.0.1 Augmentation Transform

[12]:
```
class SimCLRTransform:
    def __init__(self):
        self.transform = transforms.Compose([
            transforms.RandomResizedCrop(size=32),
            transforms.RandomHorizontalFlip(),
            transforms.RandomApply([transforms.ColorJitter(0.4, 0.4, 0.4, 0.
 ↪1)], p=0.8),
            transforms.RandomGrayscale(p=0.2),
            transforms.ToTensor(),
            transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))
        ])

    def __call__(self, x):
        return self.transform(x), self.transform(x)
```

1.0.2 ResNet-18 Encoder

[13]:
```
class ResNet18Encoder(nn.Module):
    def __init__(self, num_classes=10):
        super(ResNet18Encoder, self).__init__()
        self.resnet = models.resnet18(pretrained=False)
        self.resnet.conv1 = nn.Conv2d(3, 64, kernel_size=3, stride=1,␣
 ↪padding=1, bias=False)
        self.resnet.maxpool = nn.Identity()  # Remove maxpool layer for CIFAR-10
```

```
        self.resnet.fc = nn.Identity()  # Remove fully connected layer for␣
↪feature extraction

    def forward(self, x):
        return self.resnet(x)
```

1.0.3 Projection Head

[14]:
```
class ProjectionHead(nn.Module):
    def __init__(self, input_dim=512, hidden_dim=128, output_dim=128):
        super(ProjectionHead, self).__init__()
        self.fc1 = nn.Linear(input_dim, hidden_dim)
        self.relu = nn.ReLU()
        self.fc2 = nn.Linear(hidden_dim, output_dim)

    def forward(self, x):
        x = self.fc1(x)
        x = self.relu(x)
        x = self.fc2(x)
        return x
```

1.0.4 NT_Xent Loss

[15]:
```
def nt_xent_loss(z_i, z_j, temperature=0.5):
    batch_size = z_i.size(0)
    z = torch.cat([z_i, z_j], dim=0)  # Concatenate positive pairs

    z = nn.functional.normalize(z, dim=1)  # L2 Norm
    similarity_matrix = torch.matmul(z, z.T)/temperature  # Compute similarity␣
↪matrix

    mask = torch.eye(2 * batch_size, dtype=torch.bool).to(z.device)
    similarity_matrix = similarity_matrix[~mask].view(2 * batch_size, -1)  #␣
↪Remove self-similarity

    targets = torch.cat([
        torch.arange(batch_size -1, 2*batch_size -1),
        torch.arange(0, batch_size)
    ]).to(z.device)

    loss = nn.CrossEntropyLoss()(similarity_matrix, targets)
    return loss
```

1.0.5 Import Dataset

[16]:
```
simclr_trainset = torchvision.datasets.CIFAR10(root='./data', train=True,
                                               download=False,␣
 ↪transform=SimCLRTransform())
simclr_loader = DataLoader(simclr_trainset, batch_size=256, shuffle=True,␣
 ↪num_workers=2, drop_last=True)
```

1.0.6 Training Loop Setup

[17]:
```
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f'Using device: {device}')

# Initialize models
encoder = ResNet18Encoder().to(device)
projection_head = ProjectionHead().to(device)

# Optimizer
optimizer = torch.optim.Adam(list(encoder.parameters()) + list(projection_head.
 ↪parameters()), lr=3e-4)

# Training
num_epochs = 200
```

```
Using device: cuda
```

1.0.7 Training Loop

[19]:
```
for epoch in range(num_epochs):
    running_loss = 0.0
    loop = tqdm(simclr_loader, desc=f'Epoch [{epoch+1}/{num_epochs}]',␣
 ↪leave=False)
    for (view1, view2), _ in loop:
        view1, view2 = view1.to(device), view2.to(device)

        h_i = encoder(view1)
        h_j = encoder(view2)
        z_i = projection_head(h_i)
        z_j = projection_head(h_j)

        loss = nt_xent_loss(z_i, z_j)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        running_loss += loss.item()
        loop.set_postfix(loss=loss.item())
```

```python
    avg_loss = running_loss / len(simclr_loader)
    print(f'Epoch [{epoch+1}/{num_epochs}], Avg Loss: {avg_loss:.4f}')

# Save encoder weights
torch.save(encoder.state_dict(), 'simclr_encoder_200ep.pth')
print('Training complete. Encoder saved.')
```

```
Epoch [1/200], Avg Loss: 5.2183

Epoch [2/200], Avg Loss: 5.1332

Epoch [3/200], Avg Loss: 5.0723

Epoch [4/200], Avg Loss: 5.0247

Epoch [5/200], Avg Loss: 4.9934

Epoch [6/200], Avg Loss: 4.9643

Epoch [7/200], Avg Loss: 4.9466

Epoch [8/200], Avg Loss: 4.9254

Epoch [9/200], Avg Loss: 4.9111

Epoch [10/200], Avg Loss: 4.8974

Epoch [11/200], Avg Loss: 4.8866

Epoch [12/200], Avg Loss: 4.8787

Epoch [13/200], Avg Loss: 4.8715
```

```
Epoch [14/200], Avg Loss: 4.8661

Epoch [15/200], Avg Loss: 4.8570

Epoch [16/200], Avg Loss: 4.8472

Epoch [17/200], Avg Loss: 4.8441

Epoch [18/200], Avg Loss: 4.8370

Epoch [19/200], Avg Loss: 4.8320

Epoch [20/200], Avg Loss: 4.8296

Epoch [21/200], Avg Loss: 4.8230

Epoch [22/200], Avg Loss: 4.8189

Epoch [23/200], Avg Loss: 4.8190

Epoch [24/200], Avg Loss: 4.8123

Epoch [25/200], Avg Loss: 4.8104

Epoch [26/200], Avg Loss: 4.8063

Epoch [27/200], Avg Loss: 4.8045

Epoch [28/200], Avg Loss: 4.8027

Epoch [29/200], Avg Loss: 4.7984
```

```
Epoch [30/200], Avg Loss: 4.7955

Epoch [31/200], Avg Loss: 4.7911

Epoch [32/200], Avg Loss: 4.7882

Epoch [33/200], Avg Loss: 4.7891

Epoch [34/200], Avg Loss: 4.7852

Epoch [35/200], Avg Loss: 4.7844

Epoch [36/200], Avg Loss: 4.7781

Epoch [37/200], Avg Loss: 4.7781

Epoch [38/200], Avg Loss: 4.7788

Epoch [39/200], Avg Loss: 4.7763

Epoch [40/200], Avg Loss: 4.7715

Epoch [41/200], Avg Loss: 4.7707

Epoch [42/200], Avg Loss: 4.7711

Epoch [43/200], Avg Loss: 4.7642

Epoch [44/200], Avg Loss: 4.7661

Epoch [45/200], Avg Loss: 4.7638
```

```
Epoch [46/200], Avg Loss: 4.7609

Epoch [47/200], Avg Loss: 4.7599

Epoch [48/200], Avg Loss: 4.7613

Epoch [49/200], Avg Loss: 4.7572

Epoch [50/200], Avg Loss: 4.7559

Epoch [51/200], Avg Loss: 4.7545

Epoch [52/200], Avg Loss: 4.7496

Epoch [53/200], Avg Loss: 4.7480

Epoch [54/200], Avg Loss: 4.7510

Epoch [55/200], Avg Loss: 4.7453

Epoch [56/200], Avg Loss: 4.7469

Epoch [57/200], Avg Loss: 4.7467

Epoch [58/200], Avg Loss: 4.7419

Epoch [59/200], Avg Loss: 4.7427

Epoch [60/200], Avg Loss: 4.7389

Epoch [61/200], Avg Loss: 4.7376
```

```
Epoch [62/200], Avg Loss: 4.7425

Epoch [63/200], Avg Loss: 4.7345

Epoch [64/200], Avg Loss: 4.7355

Epoch [65/200], Avg Loss: 4.7343

Epoch [66/200], Avg Loss: 4.7300

Epoch [67/200], Avg Loss: 4.7345

Epoch [68/200], Avg Loss: 4.7332

Epoch [69/200], Avg Loss: 4.7293

Epoch [70/200], Avg Loss: 4.7292

Epoch [71/200], Avg Loss: 4.7271

Epoch [72/200], Avg Loss: 4.7246

Epoch [73/200], Avg Loss: 4.7292

Epoch [74/200], Avg Loss: 4.7210

Epoch [75/200], Avg Loss: 4.7241

Epoch [76/200], Avg Loss: 4.7213

Epoch [77/200], Avg Loss: 4.7201
```

```
Epoch [78/200], Avg Loss: 4.7221

Epoch [79/200], Avg Loss: 4.7145

Epoch [80/200], Avg Loss: 4.7214

Epoch [81/200], Avg Loss: 4.7210

Epoch [82/200], Avg Loss: 4.7186

Epoch [83/200], Avg Loss: 4.7170

Epoch [84/200], Avg Loss: 4.7145

Epoch [85/200], Avg Loss: 4.7154

Epoch [86/200], Avg Loss: 4.7124

Epoch [87/200], Avg Loss: 4.7159

Epoch [88/200], Avg Loss: 4.7120

Epoch [89/200], Avg Loss: 4.7099

Epoch [90/200], Avg Loss: 4.7127

Epoch [91/200], Avg Loss: 4.7112

Epoch [92/200], Avg Loss: 4.7077

Epoch [93/200], Avg Loss: 4.7071
```

```
Epoch [94/200], Avg Loss: 4.7070

Epoch [95/200], Avg Loss: 4.7054

Epoch [96/200], Avg Loss: 4.7065

Epoch [97/200], Avg Loss: 4.7023

Epoch [98/200], Avg Loss: 4.7059

Epoch [99/200], Avg Loss: 4.7019

Epoch [100/200], Avg Loss: 4.7026

Epoch [101/200], Avg Loss: 4.7009

Epoch [102/200], Avg Loss: 4.7004

Epoch [103/200], Avg Loss: 4.7010

Epoch [104/200], Avg Loss: 4.7020

Epoch [105/200], Avg Loss: 4.7009

Epoch [106/200], Avg Loss: 4.7000

Epoch [107/200], Avg Loss: 4.6982

Epoch [108/200], Avg Loss: 4.6958

Epoch [109/200], Avg Loss: 4.6977
```

```
Epoch [110/200], Avg Loss: 4.6968

Epoch [111/200], Avg Loss: 4.6956

Epoch [112/200], Avg Loss: 4.6936

Epoch [113/200], Avg Loss: 4.6937

Epoch [114/200], Avg Loss: 4.6965

Epoch [115/200], Avg Loss: 4.6912

Epoch [116/200], Avg Loss: 4.6901

Epoch [117/200], Avg Loss: 4.6926

Epoch [118/200], Avg Loss: 4.6934

Epoch [119/200], Avg Loss: 4.6899

Epoch [120/200], Avg Loss: 4.6902

Epoch [121/200], Avg Loss: 4.6878

Epoch [122/200], Avg Loss: 4.6898

Epoch [123/200], Avg Loss: 4.6900

Epoch [124/200], Avg Loss: 4.6900

Epoch [125/200], Avg Loss: 4.6900
```

```
Epoch [126/200], Avg Loss: 4.6847

Epoch [127/200], Avg Loss: 4.6845

Epoch [128/200], Avg Loss: 4.6839

Epoch [129/200], Avg Loss: 4.6843

Epoch [130/200], Avg Loss: 4.6863

Epoch [131/200], Avg Loss: 4.6855

Epoch [132/200], Avg Loss: 4.6832

Epoch [133/200], Avg Loss: 4.6829

Epoch [134/200], Avg Loss: 4.6819

Epoch [135/200], Avg Loss: 4.6821

Epoch [136/200], Avg Loss: 4.6788

Epoch [137/200], Avg Loss: 4.6829

Epoch [138/200], Avg Loss: 4.6806

Epoch [139/200], Avg Loss: 4.6794

Epoch [140/200], Avg Loss: 4.6772

Epoch [141/200], Avg Loss: 4.6795
```

```
Epoch [142/200], Avg Loss: 4.6773

Epoch [143/200], Avg Loss: 4.6767

Epoch [144/200], Avg Loss: 4.6746

Epoch [145/200], Avg Loss: 4.6757

Epoch [146/200], Avg Loss: 4.6758

Epoch [147/200], Avg Loss: 4.6753

Epoch [148/200], Avg Loss: 4.6750

Epoch [149/200], Avg Loss: 4.6755

Epoch [150/200], Avg Loss: 4.6764

Epoch [151/200], Avg Loss: 4.6754

Epoch [152/200], Avg Loss: 4.6719

Epoch [153/200], Avg Loss: 4.6708

Epoch [154/200], Avg Loss: 4.6723

Epoch [155/200], Avg Loss: 4.6713

Epoch [156/200], Avg Loss: 4.6734

Epoch [157/200], Avg Loss: 4.6707
```

```
Epoch [158/200], Avg Loss: 4.6717

Epoch [159/200], Avg Loss: 4.6665

Epoch [160/200], Avg Loss: 4.6690

Epoch [161/200], Avg Loss: 4.6690

Epoch [162/200], Avg Loss: 4.6679

Epoch [163/200], Avg Loss: 4.6667

Epoch [164/200], Avg Loss: 4.6649

Epoch [165/200], Avg Loss: 4.6664

Epoch [166/200], Avg Loss: 4.6680

Epoch [167/200], Avg Loss: 4.6650

Epoch [168/200], Avg Loss: 4.6660

Epoch [169/200], Avg Loss: 4.6677

Epoch [170/200], Avg Loss: 4.6673

Epoch [171/200], Avg Loss: 4.6629

Epoch [172/200], Avg Loss: 4.6677

Epoch [173/200], Avg Loss: 4.6677
```

```
Epoch [174/200], Avg Loss: 4.6660

Epoch [175/200], Avg Loss: 4.6621

Epoch [176/200], Avg Loss: 4.6654

Epoch [177/200], Avg Loss: 4.6633

Epoch [178/200], Avg Loss: 4.6596

Epoch [179/200], Avg Loss: 4.6621

Epoch [180/200], Avg Loss: 4.6591

Epoch [181/200], Avg Loss: 4.6618

Epoch [182/200], Avg Loss: 4.6606

Epoch [183/200], Avg Loss: 4.6594

Epoch [184/200], Avg Loss: 4.6593

Epoch [185/200], Avg Loss: 4.6561

Epoch [186/200], Avg Loss: 4.6621

Epoch [187/200], Avg Loss: 4.6579

Epoch [188/200], Avg Loss: 4.6597

Epoch [189/200], Avg Loss: 4.6590
```

```
Epoch [190/200], Avg Loss: 4.6591

Epoch [191/200], Avg Loss: 4.6549

Epoch [192/200], Avg Loss: 4.6548

Epoch [193/200], Avg Loss: 4.6549

Epoch [194/200], Avg Loss: 4.6559

Epoch [195/200], Avg Loss: 4.6578

Epoch [196/200], Avg Loss: 4.6572

Epoch [197/200], Avg Loss: 4.6588

Epoch [198/200], Avg Loss: 4.6559

Epoch [199/200], Avg Loss: 4.6549

Epoch [200/200], Avg Loss: 4.6554
Training complete. Encoder saved.
```

original_algorithm

March 18, 2026

1 Original TypiClust Algorithm

Reproduction of TPC_RP (SimCLR + K-Means + Typicality Selection) on CIFAR-10 with B=10.

Note: The SimCLR encoder (ResNet-18, 200 epochs) is trained from scratch in a separate notebook (`SimCLR_implementation.ipynb`). The training is split out so the algorithm can be run without retraining each time. The saved weights are loaded below.

1.1 1. Setup & Data Loading

[1]:
```
import numpy as np
import torch
import torch.nn as nn
import torchvision
import torchvision.models as models
import torchvision.transforms as transforms
from torch.utils.data import DataLoader
from sklearn.cluster import KMeans
from sklearn.neighbors import NearestNeighbors
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
```

[2]:
```
# Load CIFAR-10
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))
])

trainset = torchvision.datasets.CIFAR10(root='./data', train=True,␣
 ↪download=True, transform=transform)
testset = torchvision.datasets.CIFAR10(root='./data', train=False,␣
 ↪download=True, transform=transform)
trainloader = DataLoader(trainset, batch_size=256, shuffle=False, num_workers=2)
testloader = DataLoader(testset, batch_size=256, shuffle=False, num_workers=2)
```

1.2 2. SimCLR Encoder (Representation Learning)

[3]:
```
class ResNet18Encoder(nn.Module):
    """SimCLR encoder with ResNet-18 backbone adapted for CIFAR-10 (32x32)."""
    def __init__(self):
        super().__init__()
        self.resnet = models.resnet18(weights=None)
        self.resnet.conv1 = nn.Conv2d(3, 64, kernel_size=3, stride=1,␣
 ↪padding=1, bias=False)
        self.resnet.maxpool = nn.Identity()
        self.resnet.fc = nn.Identity()

    def forward(self, x):
        return self.resnet(x)


def load_encoder(weights_path, device='cuda'):
    encoder = ResNet18Encoder()
    encoder.load_state_dict(torch.load(weights_path, map_location=device))
    encoder = encoder.to(device)
    encoder.eval()
    return encoder


def extract_embeddings(encoder, dataloader, device='cuda'):
    encoder.eval()
    all_embeddings = []
    all_labels = []
    with torch.no_grad():
        for images, labels in dataloader:
            images = images.to(device)
            h = encoder(images)
            all_embeddings.append(h.cpu().numpy())
            all_labels.append(labels.numpy())
    return np.vstack(all_embeddings), np.hstack(all_labels)
```

[4]:
```
# Load pretrained SimCLR encoder and extract embeddings
encoder = load_encoder('./simclr_encoder_200ep.pth')
embeddings, labels = extract_embeddings(encoder, trainloader)
test_embeddings, test_labels = extract_embeddings(encoder, testloader)
print(f"Train embeddings: {embeddings.shape}")
print(f"Test embeddings: {test_embeddings.shape}")
```

```
Train embeddings: (50000, 512)
Test embeddings: (10000, 512)
```

1.3 3. Typicality Computation

[5]:
```
def compute_typicality(embeddings, k=20):
    """Compute typicality as inverse mean distance to k nearest neighbours."""
    nn = NearestNeighbors(n_neighbors=k + 1)
    nn.fit(embeddings)
    distances, _ = nn.kneighbors(embeddings)
    distances = distances[:, 1:]  # exclude self
    return np.mean(distances, axis=1) ** -1


typicality = compute_typicality(embeddings, k=20)
print(f"Typicality shape: {typicality.shape}")
print(f"Typicality range: [{typicality.min():.4f}, {typicality.max():.4f}]")
```

```
Typicality shape: (50000,)
Typicality range: [0.0637, 0.4876]
```

1.4 4. Clustering (K-Means, K=B)

[6]:
```
def cluster_standard(embeddings, budget, random_state=42):
    """K-Means clustering with K = budget."""
    kmeans = KMeans(n_clusters=budget, random_state=random_state, n_init=10)
    kmeans.fit(embeddings)
    return kmeans.labels_


budget = 10
cluster_labels = cluster_standard(embeddings, budget)
print(f"Cluster sizes: {np.bincount(cluster_labels)}")
```

```
Cluster sizes: [4518 4837 9152 4655 6773 5676 4402 4942 3035 2010]
```

1.5 5. Selection (Max Typicality)

[7]:
```
def select_max_typicality(typicality, cluster_labels, budget):
    """Select the most typical point from each cluster."""
    selected = []
    for cluster in range(budget):
        cluster_idx = np.where(cluster_labels == cluster)[0]
        best = cluster_idx[np.argmax(typicality[cluster_idx])]
        selected.append(best)
    return np.array(selected)


selected_indices = select_max_typicality(typicality, cluster_labels, budget)
print(f"Selected indices: {selected_indices}")
print(f"Selected labels: {labels[selected_indices]}")
```

```
print(f"Classes covered: {len(np.unique(labels[selected_indices]))}")
```

```
Selected indices: [33063 44775  5887 48525 41454 16434 49134 32286 29916 37990]
Selected labels: [1 9 7 6 2 4 8 7 7 2]
Classes covered: 7
```

1.6 6. Evaluation (Linear Probe)

[8]:
```
def evaluate_selection(selected_indices, embeddings, labels, test_embeddings,␣
↪test_labels):
    """Train a linear probe on selected examples and evaluate on test set."""
    sel_emb = embeddings[selected_indices]
    sel_lab = labels[selected_indices]
    clf = LogisticRegression(max_iter=1000)
    clf.fit(sel_emb, sel_lab)
    preds = clf.predict(test_embeddings)
    accuracy = accuracy_score(test_labels, preds)
    n_classes = len(np.unique(sel_lab))
    return {'accuracy': accuracy, 'n_classes_covered': n_classes}


results = evaluate_selection(selected_indices, embeddings, labels,␣
↪test_embeddings, test_labels)
print(f"\n=== Original TypiClust Results ===")
print(f"Accuracy: {results['accuracy']:.4f}")
print(f"Classes covered: {results['n_classes_covered']}/10")
```

```
=== Original TypiClust Results ===
Accuracy: 0.4476
Classes covered: 7/10
```

1.7 7. Random Baseline Comparison

[9]:
```python
# Random baseline (30 seeds)
random_accs = []
for seed in range(30):
    rng = np.random.RandomState(seed)
    idx = rng.choice(len(embeddings), size=budget, replace=False)
    clf = LogisticRegression(max_iter=1000)
    clf.fit(embeddings[idx], labels[idx])
    preds = clf.predict(test_embeddings)
    random_accs.append(accuracy_score(test_labels, preds))

random_accs = np.array(random_accs)
print(f"Random Baseline: {random_accs.mean():.4f} +/- {random_accs.std():.4f}")
print(f"TypiClust: {results['accuracy']:.4f}")
print(f"Improvement: +{results['accuracy'] - random_accs.mean():.4f}")
```

```
Random Baseline: 0.3692 +/- 0.0675
TypiClust: 0.4476
Improvement: +0.0784
```

modified_algorithm

March 18, 2026

1 Modified TypiClust Algorithm

Best configuration: UMAP (n=5) + Hybrid Selection (alpha=0.2) on CIFAR-10 with B=10.

1.1 1. Setup & Data Loading

[1]:
```
import numpy as np
import torch
import torch.nn as nn
import torchvision
import torchvision.models as models
import torchvision.transforms as transforms
from torch.utils.data import DataLoader
from sklearn.cluster import KMeans
from sklearn.neighbors import NearestNeighbors
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
import umap
```

[2]:
```
# Load CIFAR-10
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))
])

trainset = torchvision.datasets.CIFAR10(root='./data', train=True,␣
 ↪download=True, transform=transform)
testset = torchvision.datasets.CIFAR10(root='./data', train=False,␣
 ↪download=True, transform=transform)
trainloader = DataLoader(trainset, batch_size=256, shuffle=False, num_workers=2)
testloader = DataLoader(testset, batch_size=256, shuffle=False, num_workers=2)
```

1.2 2. SimCLR Encoder (Representation Learning)

[3]:
```
class ResNet18Encoder(nn.Module):
    """SimCLR encoder with ResNet-18 backbone adapted for CIFAR-10 (32x32)."""
    def __init__(self):
```

```
        super().__init__()
        self.resnet = models.resnet18(weights=None)
        self.resnet.conv1 = nn.Conv2d(3, 64, kernel_size=3, stride=1,␣
↪padding=1, bias=False)
        self.resnet.maxpool = nn.Identity()
        self.resnet.fc = nn.Identity()

    def forward(self, x):
        return self.resnet(x)


def load_encoder(weights_path, device='cuda'):
    encoder = ResNet18Encoder()
    encoder.load_state_dict(torch.load(weights_path, map_location=device))
    encoder = encoder.to(device)
    encoder.eval()
    return encoder


def extract_embeddings(encoder, dataloader, device='cuda'):
    encoder.eval()
    all_embeddings = []
    all_labels = []
    with torch.no_grad():
        for images, labels in dataloader:
            images = images.to(device)
            h = encoder(images)
            all_embeddings.append(h.cpu().numpy())
            all_labels.append(labels.numpy())
    return np.vstack(all_embeddings), np.hstack(all_labels)
```

[4]:
```
# Load pretrained SimCLR encoder and extract embeddings
encoder = load_encoder('./simclr_encoder_200ep.pth')
embeddings, labels = extract_embeddings(encoder, trainloader)
test_embeddings, test_labels = extract_embeddings(encoder, testloader)
print(f"Train embeddings: {embeddings.shape}")
print(f"Test embeddings: {test_embeddings.shape}")
```

```
Train embeddings: (50000, 512)
Test embeddings: (10000, 512)
```

1.3 3. UMAP Dimensionality Reduction (Modification 1)

[5]:
```
def preprocess_umap(embeddings, n_components=5, random_state=42):
    """Reduce embedding dimensionality with UMAP before clustering."""
    reducer = umap.UMAP(n_components=n_components, random_state=random_state)
    transformed = reducer.fit_transform(embeddings)
```

```
    return transformed, reducer


# Apply UMAP (n=5) to train and test embeddings
proc_embeddings, umap_model = preprocess_umap(embeddings, n_components=5)
proc_test = umap_model.transform(test_embeddings)
print(f"UMAP train embeddings: {proc_embeddings.shape}")
print(f"UMAP test embeddings: {proc_test.shape}")
```

```
/mnt/data/conda/envs/ml-cw2/lib/python3.13/site-packages/umap/umap_.py:1952:
UserWarning: n_jobs value 1 overridden to 1 by setting random_state. Use no seed
for parallelism.
  warn(

UMAP train embeddings: (50000, 5)
UMAP test embeddings: (10000, 5)
```

1.4 4. Typicality Computation

[6]:
```
def compute_typicality(embeddings, k=20):
    """Compute typicality as inverse mean distance to k nearest neighbours."""
    nn = NearestNeighbors(n_neighbors=k + 1)
    nn.fit(embeddings)
    distances, _ = nn.kneighbors(embeddings)
    distances = distances[:, 1:]  # exclude self
    return np.mean(distances, axis=1) ** -1


typicality = compute_typicality(proc_embeddings, k=20)
print(f"Typicality shape: {typicality.shape}")
print(f"Typicality range: [{typicality.min():.4f}, {typicality.max():.4f}]")
```

```
Typicality shape: (50000,)
Typicality range: [1.4265, 49.2716]
```

1.5 5. Clustering (K-Means, K=B)

[7]:
```
def cluster_standard(embeddings, budget, random_state=42):
    """K-Means clustering with K = budget."""
    kmeans = KMeans(n_clusters=budget, random_state=random_state, n_init=10)
    kmeans.fit(embeddings)
    return kmeans.labels_


budget = 10
cluster_labels = cluster_standard(proc_embeddings, budget)
print(f"Cluster sizes: {np.bincount(cluster_labels)}")
```

```
Cluster sizes: [5585 7169 4816 3861 4940 2399 4925 5329 6171 4805]
```

1.6 6. Hybrid Selection (Modification 2)

[8]:
```
def select_hybrid(embeddings, typicality, cluster_labels, budget, alpha=0.2):
    """
    Hybrid selection: score = alpha * typicality + (1-alpha) * min_distance.
    Selects sequentially, balancing representativeness with spatial diversity.
    """
    selected = []

    # Order clusters by max typicality (most typical cluster first)
    cluster_order = []
    for c in range(budget):
        cluster_idx = np.where(cluster_labels == c)[0]
        cluster_order.append((c, np.max(typicality[cluster_idx])))
    cluster_order.sort(key=lambda x: x[1], reverse=True)

    # First point: most typical overall
    first_cluster = cluster_order[0][0]
    cluster_idx = np.where(cluster_labels == first_cluster)[0]
    best = cluster_idx[np.argmax(typicality[cluster_idx])]
    selected.append(best)

    # Subsequent points: hybrid score within each cluster
    for cluster in cluster_order[1:]:
        cluster_idx = np.where(cluster_labels == cluster[0])[0]
        candidate_embeddings = embeddings[cluster_idx]

        # Compute min distance to already-selected points
        dist = np.linalg.norm(
            candidate_embeddings[:, None] - embeddings[selected], axis=2
        )
        min_distance = dist.min(axis=1)

        # Normalise both terms to [0, 1] within this cluster
        norm_distance = (min_distance - min_distance.min()) / (
            min_distance.max() - min_distance.min() + 1e-8
        )
        norm_typicality = (typicality[cluster_idx] - typicality[cluster_idx].
↪min()) / (
            typicality[cluster_idx].max() - typicality[cluster_idx].min() + 1e-8
        )

        # Hybrid score
        score = alpha * norm_typicality + (1 - alpha) * norm_distance
        best = cluster_idx[np.argmax(score)]
        selected.append(best)
```

```
    return np.array(selected)


selected_indices = select_hybrid(proc_embeddings, typicality, cluster_labels,␣
 ↪budget, alpha=0.2)
print(f"Selected indices: {selected_indices}")
print(f"Selected labels: {labels[selected_indices]}")
print(f"Classes covered: {len(np.unique(labels[selected_indices]))}")
```

```
Selected indices: [11148 39670  5922  8907 11196 24391 34846  3866 18610  6492]
Selected labels: [8 0 2 7 1 9 5 4 6 3]
Classes covered: 10
```

1.7 7. Evaluation (Linear Probe)

[9]:
```
def evaluate_selection(selected_indices, embeddings, labels, test_embeddings,␣
 ↪test_labels):
    """Train a linear probe on selected examples and evaluate on test set."""
    sel_emb = embeddings[selected_indices]
    sel_lab = labels[selected_indices]
    clf = LogisticRegression(max_iter=1000)
    clf.fit(sel_emb, sel_lab)
    preds = clf.predict(test_embeddings)
    accuracy = accuracy_score(test_labels, preds)
    n_classes = len(np.unique(sel_lab))
    return {'accuracy': accuracy, 'n_classes_covered': n_classes}


results = evaluate_selection(selected_indices, proc_embeddings, labels,␣
 ↪proc_test, test_labels)
print(f"\n=== Modified TypiClust Results (UMAP-5 + Hybrid alpha=0.2) ===")
print(f"Accuracy: {results['accuracy']:.4f}")
print(f"Classes covered: {results['n_classes_covered']}/10")
```

```
=== Modified TypiClust Results (UMAP-5 + Hybrid alpha=0.2) ===
Accuracy: 0.6949
Classes covered: 10/10
```

1.8 8. Comparison with Baseline

[10]:
```
# Random baseline (30 seeds) on UMAP-reduced embeddings
random_accs = []
for seed in range(30):
    rng = np.random.RandomState(seed)
    idx = rng.choice(len(proc_embeddings), size=budget, replace=False)
    clf = LogisticRegression(max_iter=1000)
    clf.fit(proc_embeddings[idx], labels[idx])
```

```python
    preds = clf.predict(proc_test)
    random_accs.append(accuracy_score(test_labels, preds))

random_accs = np.array(random_accs)

# Original TypiClust baseline (no UMAP, no hybrid) on raw embeddings
from sklearn.cluster import KMeans as KM
km = KM(n_clusters=budget, random_state=42, n_init=10)
km.fit(embeddings)
baseline_typ = compute_typicality(embeddings, k=20)
baseline_sel = []
for c in range(budget):
    cidx = np.where(km.labels_ == c)[0]
    baseline_sel.append(cidx[np.argmax(baseline_typ[cidx])])
baseline_sel = np.array(baseline_sel)
clf = LogisticRegression(max_iter=1000)
clf.fit(embeddings[baseline_sel], labels[baseline_sel])
baseline_acc = accuracy_score(test_labels, clf.predict(test_embeddings))

print(f"Random Baseline: {random_accs.mean():.4f} +/- {random_accs.std():.4f}")
print(f"Original TypiClust: {baseline_acc:.4f}")
print(f"Modified (UMAP-5 + Hybrid): {results['accuracy']:.4f}")
print(f"Improvement over baseline: +{results['accuracy'] - baseline_acc:.4f}")
```

```
Random Baseline: 0.4448 +/- 0.0722
Original TypiClust: 0.4476
Modified (UMAP-5 + Hybrid): 0.6949
Improvement over baseline: +0.2473
```